

Table of Contents

Davis & Shirtliff — Ultimate God Mode Architecture (Dynamic, Modular & Multi-Cloud)

Scaffolding-first • Repository contract • 3 mature tiers • Dynamic Targets • Dynamic Secrets • Multi-Cloud (AWS/GCP/Azure) • Concurrency-safe • External contributor flow • 24-hour approval window • Separated observability

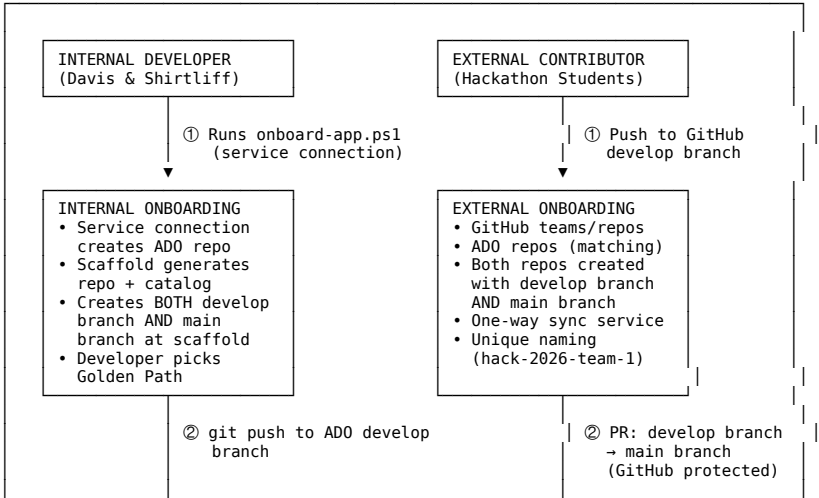
Corrections Applied to This Version (Expert Review + Multi-Cloud Expansions)

#	Correction	Before	After
1	Branch model	Pipeline creates main branch after prod deployment (unusual)	Standard model: develop branch → PR → main branch → production. Main branch is the source of truth, not a deployment artifact
2	Approval gate	“24-hour auto-approval” (dangerous wording)	“24-hour approval window with automatic cancellation on timeout” — never interprets a timeout as approval
3	Repo validation	“Only src/ + tests/ allowed” (too restrictive)	Repository contract: required / allowed / forbidden lists. Allows docs/, migrations/, language manifests, openapi; forbids pipeline definitions, secrets, .env, k8s manifests, Helm charts
4	Secrets model	Hardcoded Azure Key Vault	Dynamic Secrets Provider Abstraction. Catalog declares secretsProvider (Azure KV, HashiCorp Vault, AWS Secrets Manager, CyberArk Conjur, Akeyless). Platform routes to correct provider module
5	DR model	“Data + VM replicas” (incomplete)	Three explicit layers: High Availability (replicas/zones),

#	Correction	Before	After
6	Developer experience	PowerShell forever	Disaster Recovery (secondary region + geo-replication), Cyber-resilience (immutable backups, isolated access, quarterly recovery tests) Evolution path: PowerShell (Phase 1) → Platform API (Phase 2) → Developer Portal (Phase 3 end-state)
7	Observability	Grafana as sole source of truth	Three separated concerns: Operational observability (Grafana/Prometheus/Cloud Monitors), Developer feedback (ADO PR checks/notifications), Security governance (Defender/Sentinel/Policy)
8	Deployment Targets	Hardcoded Azure AKS	Dynamic Deployment Target Abstraction. Catalog declares deploymentTarget (AKS, EKS, GKE, DigitalOcean, Firebase, Supabase, AWS Amplify, Kamatera, Hostinger, etc.). Supports K8s, PaaS, BaaS, and VMs
9	Non-K8s Workloads	Assumed Kubernetes/Helm	Provider-agnostic deploy step. Pipeline routes to native CLIs/APIs (firebase deploy, supabase db push, doctl, AWS CDK) based on target. Mobile apps and static sites fully supported
10	Extensibility	Monolithic Terraform/Helm	Modular Provider Plugin Architecture. New tech stack = add folder to providers/compute/ or providers/secrets/. Core Spine never changes. Freedom of choice for any

#	Correction	Before	After
11	Branch naming	“develop” and “main”	tech stack Explicitly “ develop branch ” and “ main branch ” everywhere. Only 2 branches exist per repo
12	Multi-Cloud Routing	Implicit cloud support	Explicit Multi-Cloud Routing & Security. AWS EKS (IRSA), GCP GKE (Workload Identity), and Azure AKS (Entra WI) are fully supported via identical pipeline spine and dynamic provider plugins
13	Dynamic Observability	Hardcoded Azure Monitor	Dynamic Observability Provider. Catalog declares observabilityProvider (Azure Monitor, CloudWatch, Cloud Logging, Datadog, etc.) routing metrics to the correct cloud-native or 3rd-party stack
14	Target Environments	Prod/DR only in registry	Staging, Prod, and DR explicitly mapped for all clouds. Kubernetes is strictly optional per cloud (AWS App Runner, GCP Cloud Run, Azure App Service supported alongside EKS/GKE/AKS)

DIAGRAM 1 — The North Star (Total System: Internal + External + Multi-Cloud)



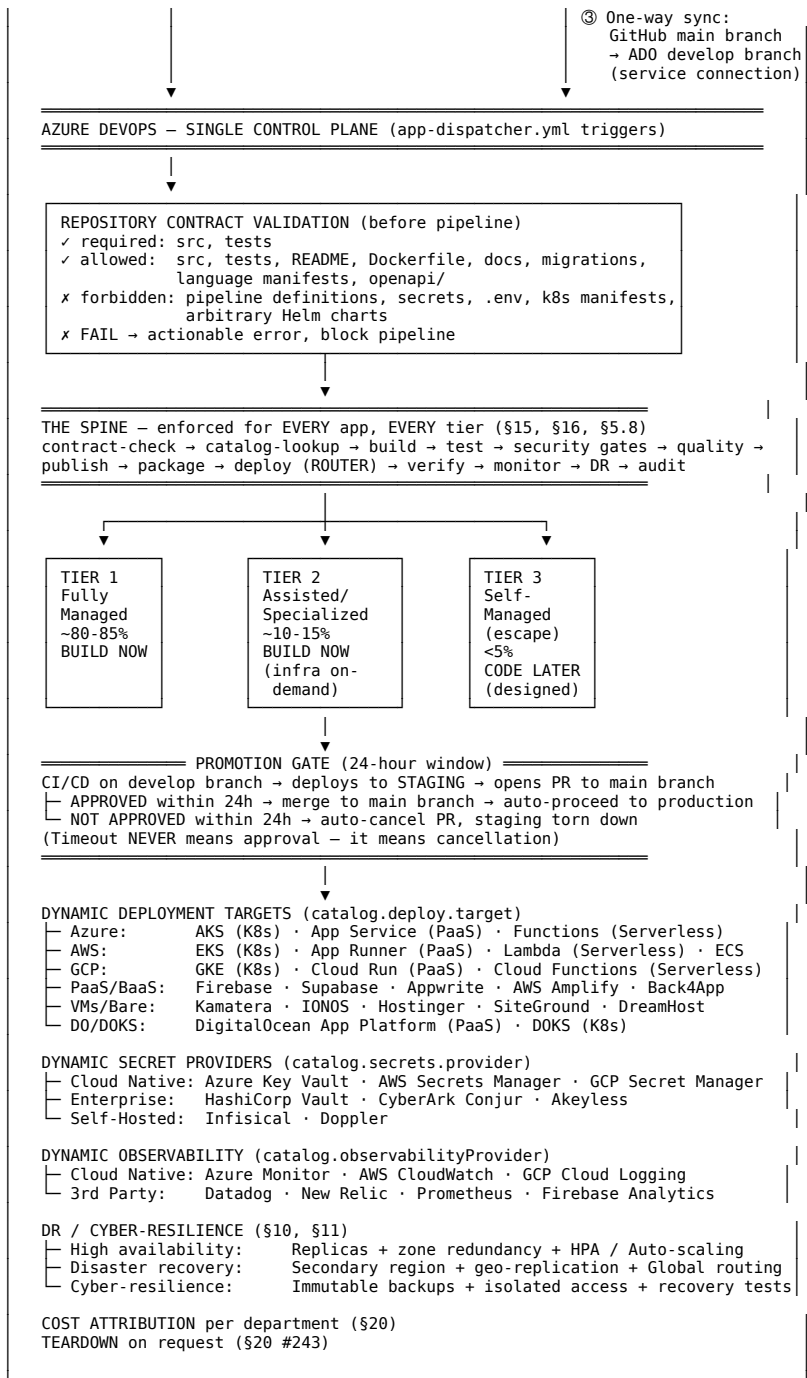
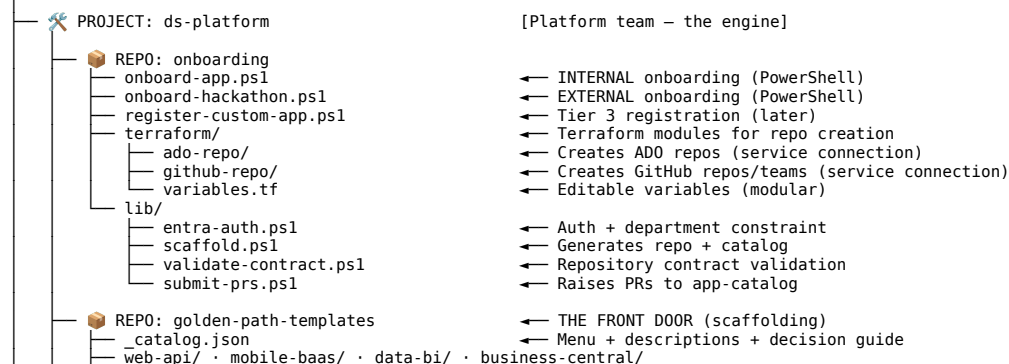


DIAGRAM 2 — Complete Repository Structure (Internal + External, Modular Providers)

D&S Azure DevOps (Organization)



infrastructure/ · peripheral-software/	← TIER 1 – standard paths
specialized/	← TIER 2 – specialized paths
— ai-ml-training/ · streaming/ · edge/	
 REPO: pipeline-templates	
app-dispatcher.yml	← Master pipeline (all apps extend)
platform-policy.yml	← THE BRAIN (derivation rules)
shared/	← THE SPINE (all tiers)
repo-contract-check.yml	← Validates required/allowed/forbidden
catalog-lookup.yml	← Reads catalog (goldenPath known)
ci-base.yml	← Build + tests
devsecops-gates.yml	← SAST/SCA/secret/container/SBOM/IaC
quality-gates.yml	← Coverage/smells/license
publish-results.yml	← → Prometheus/Log Analytics → Grafana
package.yml	← Build artifact (image, bundle, zip)
deploy-router.yml	← ROUTES to correct provider module
verify.yml	← Health checks + smoke tests
monitor-registry.yml	← Register Observability Provider
dr-provision.yml	← Terraform HA + DR + cyber-resilience
dr-teardown.yml	← Destroy replicas on request
golden-paths/	← Deploy profiles
web-api.yml · mobile-baas.yml · data-bi.yml	
business-central.yml · infrastructure.yml · peripheral-software.yml	
specialized/	
— ai-ml-training.yml · streaming.yml · edge.yml	
escape-hatch/	← TIER 3 (code later)
escape-hatch.yml	← Trigger team pipeline + enforce spine
contract-verify.yml	← Verify results/observability/cost
 REPO: providers	← THE EXTENSIBILITY ENGINE (Modular)
compute/	← Deployment Targets
azure-aks/	← Terraform + Helm (K8s)
deploy.yml	← Helm + CSI driver
workload-identity.yml	← Entra federated credential
terraform/	← AKS cluster provisioning
azure-app-service/	← Azure CLI / Bicep (PaaS, NO K8s)
azure-functions/	← Azure CLI (Serverless, NO K8s)
aws-eks/	← AWS PLUGIN (K8s)
deploy.yml	← Helm + EKS CSI driver
iam-role-for-service-accounts.yml	← IRSA setup
terraform/	
— main.tf	← EKS cluster module
— irsa.tf	← OIDC provider + IAM roles
— variables.tf	
README.md	
aws-app-runner/	← AWS CLI / CDK (PaaS, NO K8s)
aws-lambda/	← AWS SAM / CDK (Serverless, NO K8s)
gcp-gke/	← GCP PLUGIN (K8s)
deploy.yml	← Helm + GKE CSI driver
workload-identity.yml	← GKE Workload Identity
terraform/	
— main.tf	← GKE cluster module
— workload-identity.tf	← GSA ↔ KSA binding
— variables.tf	
README.md	
gcp-cloud-run/	← gcloud CLI (PaaS, NO K8s)
gcp-cloud-functions/	← gcloud CLI (Serverless, NO K8s)
digitalocean-app/	← Terraform + doctl (PaaS, NO K8s)
digitalocean-doks/	← Terraform + Helm (K8s)
firebase/	← Terraform + firebase CLI (BaaS)
supabase/	← Terraform + supabase CLI (BaaS)
aws-amplify/	← Terraform + AWS CDK (PaaS)
kamatera-vm/	← Terraform + SSH/Ansible (VM)
ionos-vm/ · hostinger/ · siteground/	← Any provider you add
... (modular, add any tech stack)	
secrets/	← Secret Providers
azure-key-vault/	← CSI driver + Workload Identity
aws-secrets-manager/	← AWS PLUGIN
inject.yml	← IRSA + CSI
terraform/	
gcp-secret-manager/	← GCP PLUGIN
inject.yml	← Workload Identity + CSI
terraform/	
hashicorp-vault/	← Vault Agent Injector
cyberark-conjur/	← Conjur Auth Provider
akeyless/	← Akeyless API
observability/	← Observability Providers (Dynamic)
azure-monitor/ · cloudwatch/ · cloud-logging/	
datadog/ · newrelic/ · firebase-analytics/	
 REPO: target-registry	← Replaces cluster-registry
targets-ci.yml	← Validates targets on PR
targets.yml	← Maps targets to provider configs
 REPO: app-catalog	[grouped by 8 ICT DEPARTMENTS]
catalog-ci.yml	← Validates entries on PR
catalog-schema.json	← The contract
catalog/	
1-digital-channels/ · 2-business-central/	
3-infrastructure/ · 4-business-intelligence/	
5-ai-sizing/ · 6-peripheral-software/	
7-operations/ · 8-governance/	
<project>---<app>.yaml	← Tiny per-app entries
 REPO: observability	
grafana-dashboards/	← Engineering observability only
alerting-rules/	
provisioning/	

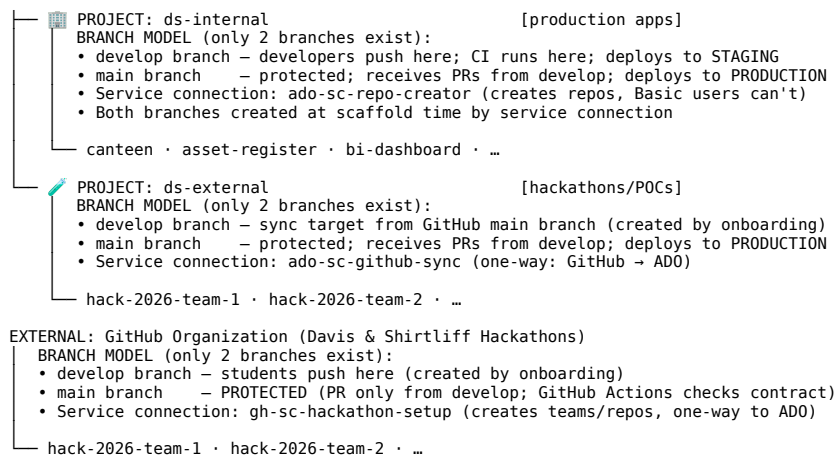
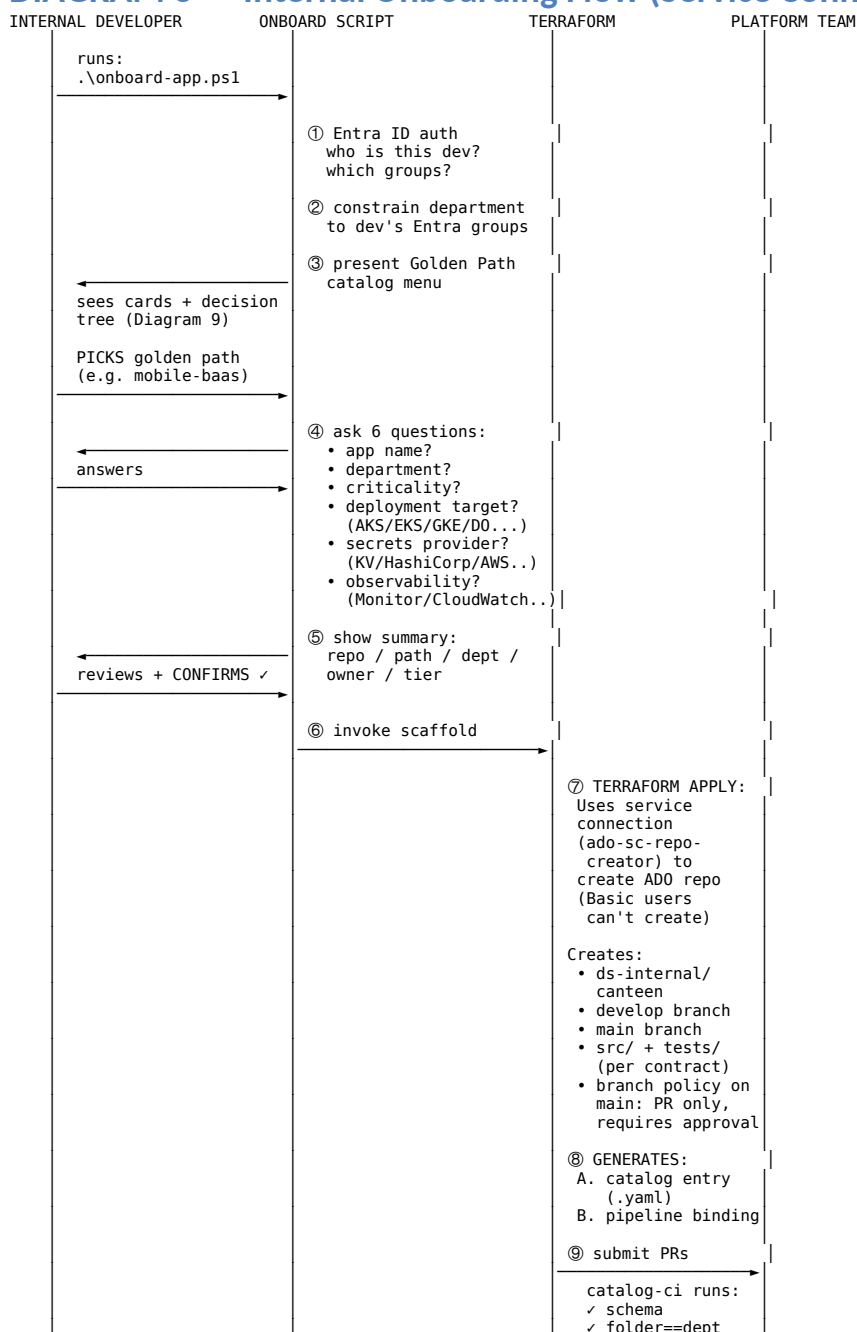
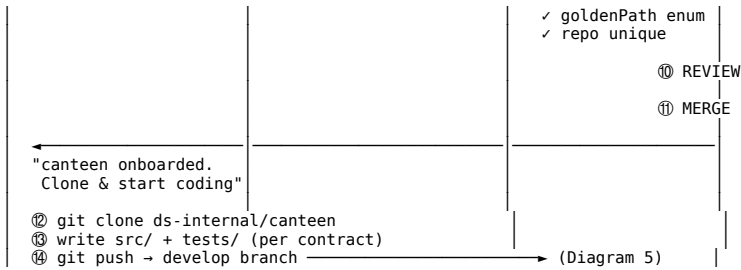


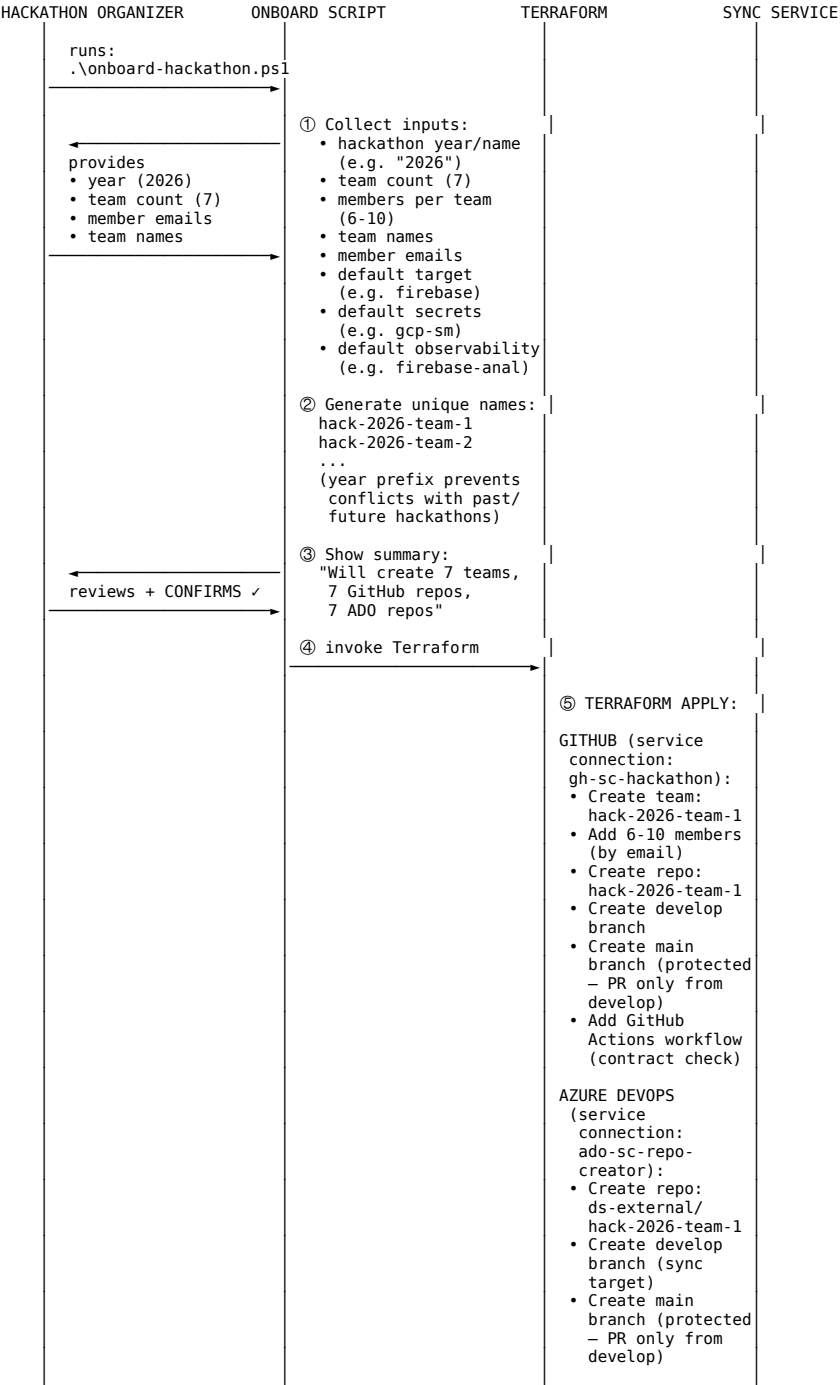
DIAGRAM 3 — Internal Onboarding Flow (Service Connection, Scaffold)

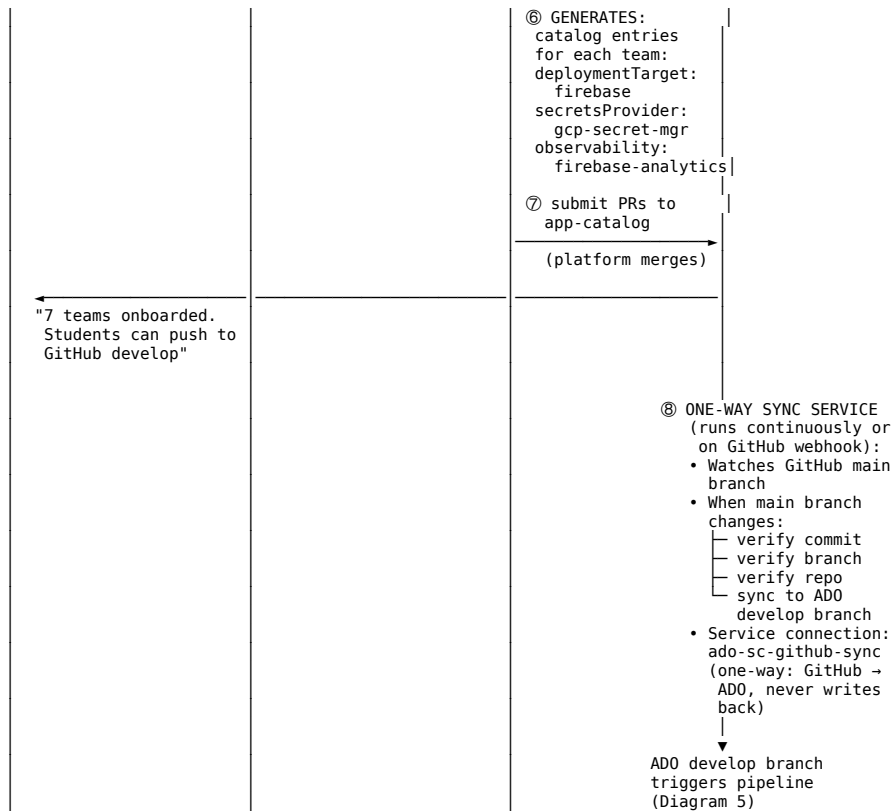




Key: Service connection creates the repo (developer has Basic license). Both develop branch and main branch are created at scaffold time. Main branch is protected (PR only from develop branch, requires approval). Developer selects target, secrets provider, and observability provider dynamically.

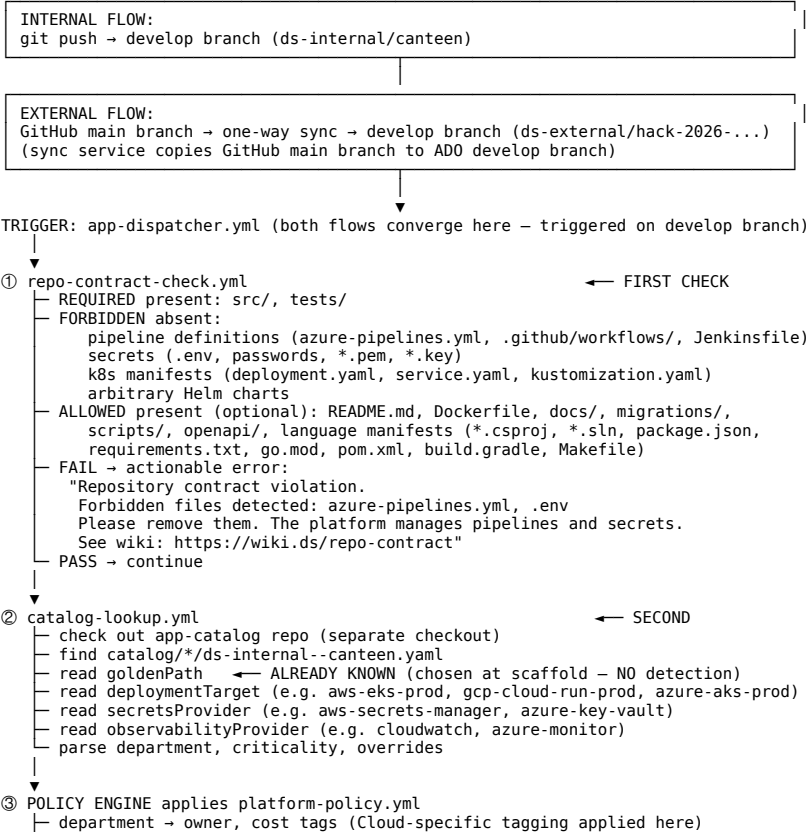
DIAGRAM 4 — External Onboarding Flow (Hackathon: GitHub + ADO)

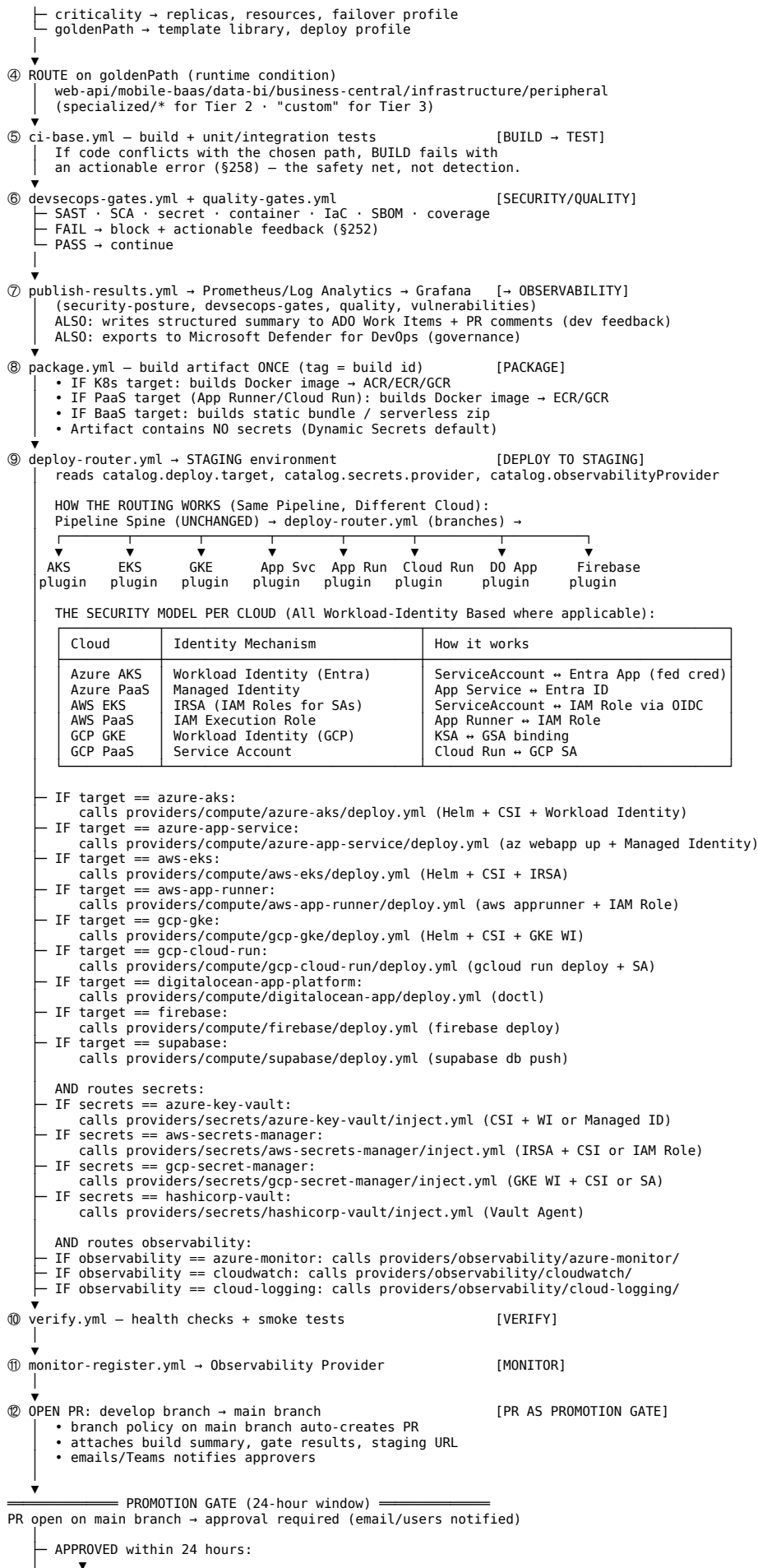


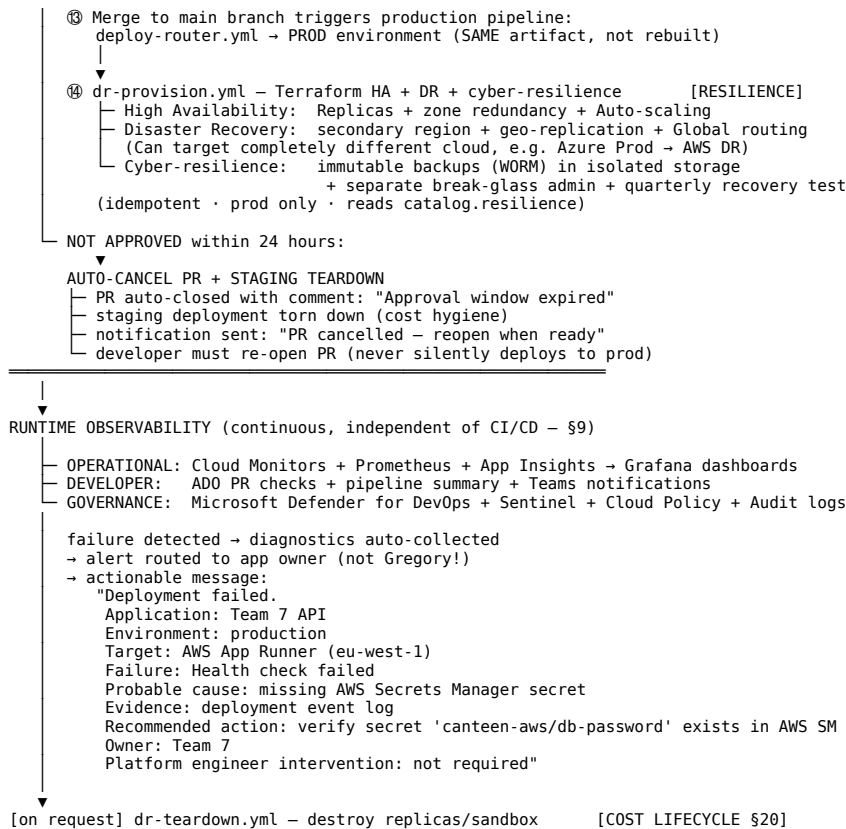


Key: External onboarding creates both GitHub and ADO repos with matching names (hack-2026-team-1). GitHub has protected main branch (PR only). One-way sync service copies GitHub main branch → ADO develop branch. Students never touch ADO directly. Hackathons can default to BaaS targets like Firebase/Supabase.

DIAGRAM 5 — Workflow from git push to Production (Internal + External + Multi-Cloud)







Key Changes in This Diagram: 1. **Repository contract** (required/allowed/forbidden) replaces strict “src/+tests/ only” 2. **Branch model is standard:** develop branch – CI – staging – PR to main branch – production 3. **24-hour approval window** with auto-cancel on timeout (never auto-approves) 4. **Dynamic Targets, Secrets & Observability** explicitly routed in Step ⑩ (deploy-router.yml), now including AWS EKS and GCP GKE 5. **Multi-Cloud Routing Mechanics & Security Models** (IRSA, GKE WI, PaaS Managed Identities) explicitly detailed in Step ⑩ 6. **Kubernetes is strictly optional:** PaaS and Serverless routing options provided for AWS, GCP, Azure, and DO 7. **Non-K8s workloads** fully supported via provider-specific CLIs/APIs 8. **DR expanded to three layers:** HA, Disaster Recovery (cross-cloud supported), Cyber-resilience 9. **Observability separated** into three concerns (operational/developer/governance) 10. **Failure diagnostics** give actionable info to app owner (not platform team)

DIAGRAM 6 — Catalog .yaml for Each Tier (Dynamic Providers & Multi-Cloud)

Tier 1 — Azure AKS + Azure Key Vault + Azure Monitor

```
# catalog/6-peripheral-software/ds-internal--canteen-azure.yaml
department: peripheral-software
criticality: high
goldenPath: web-api
deploymentTarget: azure-aks-prod          # ← Azure K8s
secretsProvider: azure-key-vault          # ← Azure
observabilityProvider: azure-monitor       # ← Dynamic Observability

config:
  ingress: { host: canteen.dayliff.com }
  database: { type: azure-sql, sku: S1 }
secrets:
  - canteen-db-password
  - canteen-jwt-signing-key
```

Tier 1 — AWS App Runner (NO Kubernetes) + AWS Secrets Manager + CloudWatch

```
# catalog/6-peripheral-software/ds-internal--canteen-aws-paas.yaml
department: peripheral-software
criticality: high
goldenPath: web-api
deploymentTarget: aws-app-runner-prod     # ← AWS PaaS (NO K8s required)
secretsProvider: aws-secrets-manager      # ← AWS
observabilityProvider: cloudwatch         # ← Dynamic Observability

config:
```

```

domain: canteen-aws-paas.dayliff.com
instanceType: cpu-1-mem-2
secrets:
  - canteen-aws/db-password
  - canteen-aws/jwt-signing-key

```

Tier 1 — GCP Cloud Run (NO Kubernetes) + GCP Secret Manager + Cloud Logging

```

# catalog/6-peripheral-software/ds-internal--canteen-gcp-paas.yaml
department: peripheral-software
criticality: high
goldenPath: web-api
deploymentTarget: gcp-cloud-run-prod # ← GCP PaaS (NO K8s required)
secretsProvider: gcp-secret-manager # ← GCP
observabilityProvider: cloud-logging # ← Dynamic Observability

```

```

config:
  domain: canteen-gcp-paas.dayliff.com
  concurrency: 80
secrets:
  - projects/ds-prod/secrets/canteen-db-password
  - projects/ds-prod/secrets/canteen-jwt-signing-key

```

Tier 1 — Azure App Service (NO Kubernetes)

```

# catalog/6-peripheral-software/ds-internal--canteen-azure-paas.yaml
department: peripheral-software
criticality: high
goldenPath: web-api
deploymentTarget: azure-appsvc-prod # ← Azure PaaS (NO K8s required)
secretsProvider: azure-key-vault
observabilityProvider: azure-monitor

```

```

config:
  host: canteen-appsvc.dayliff.com
  sku: B1
secrets:
  - canteen-db-password

```

All three use the exact same pipeline spine, same gates, same repository contract, same 24-hour promotion window. Kubernetes is NOT mandatory for any cloud.

Tier 1 — True Multi-Cloud DR (Azure Primary + AWS DR)

```

# catalog/2-business-central/ds-internal--bc-backup.yaml
department: business-central
criticality: critical
goldenPath: backup-system

# Primary: Azure App Service
deploymentTarget: azure-appsvc-prod

# DR: AWS App Runner (completely independent cloud, NO K8s needed)
drTarget: aws-app-runner-prod

secretsProvider: azure-key-vault # Primary secrets
drSecretsProvider: aws-secrets-manager # DR secrets (synced securely)
observabilityProvider: datadog # 3rd party aggregator for multi-cloud view

resilience:
  ha: true
  dr: true # Auto-failover to AWS if Azure fails
  cyberResilience: true # Immutable backups in both clouds

```

If Azure has a regional outage → Front Door / Global routing → AWS App Runner takes over automatically. Your data is replicated, your app is identical, your secrets are safely synced. No single cloud vendor can take you down.

Tier 1 — Non-K8s Mobile BaaS (Firebase + GCP Secret Manager)

```

# catalog/1-digital-channels/ds-internal--customer-app.yaml
department: digital-channels
criticality: medium
goldenPath: mobile-baas
deploymentTarget: firebase # Routes to providers/compute/firebase
secretsProvider: gcp-secret-manager # Routes to providers/secrets/gcp-secret-manager
observabilityProvider: firebase-analytics

config:
  projectId: ds-customer-app-prod
  services: [auth, firestore, storage]

secrets:
  - firebase-admin-sdk
  - stripe-publishable-key

```

No Kubernetes required. Pipeline uses firebase deploy natively.

Tier 2 — Assisted / Specialized

```

# catalog/5-ai-sizing/ds-internal--pump-sizing-training.yaml
department: ai-sizing
criticality: high

```

```
goldenPath: ai-ml-training
deploymentTarget: azure-aks-gpu
secretsProvider: azure-key-vault
observabilityProvider: azure-monitor

workload:                                # team-supplied (platform can't default)
  framework: pytorch
  gpuType: nvidia-t4
  gpuCount: 2
  distributed: true
  trainingData: { source: azure-blob, container: pump-sizing-datasets }
  output: { modelRegistry: mlflow, artifactPath: models/pump-sizing/ }
```

Specialized infra (aks-gpu-01) provisioned **on-demand** when this merges.

Tier 3 — Self-Managed / Escape Hatch (designed mature, code later)

```
# catalog/1-digital-channels/ds-internal--payment-gateway.yaml
department: digital-channels
criticality: high
goldenPath: custom                                # escape hatch

custom:
  pipelineRepo: ds-internal/payment-gateway-pipeline
  compliance: pci-dss
  deploymentTarget: kamatera-vm                    # custom VM provider
  secretsProvider: cyberark-conjur                 # enterprise secrets
  observabilityProvider: datadog
  contracts:                                       # what the team must satisfy
    publishSecurityResults: true
    registerObservability: true
    applyCostTags: true
  reReviewDate: 2027-02-13                        # 6-month re-review
```

	Tier 1	Tier 2	Tier 3
Catalog generated by	scaffold (auto)	scaffold (auto)	registration/approval
Repo created?	✓	✓	✗ (team owns)
Pipeline binding	app-dispatcher	app-dispatcher	team’s own + spine
Dynamic Targets	✓ (any provider)	✓ (any provider)	✓ (any provider)
Approval gate	routine merge	routine merge	explicit approval
Build now?	✓	✓	code later

DIAGRAM 7 — Dynamic Target & Provider Registries + Guardrails + Cost

ds-platform/target-registry/targets.yml ← fully dynamic, append-only

```
targets:
  # — AZURE (K8s & PaaS – Staging, Prod, DR) —
  azure-aks-staging: { provider: azure-aks,      serviceConnection: ado-sc-azure-staging, resource: aks-stg-01, cloud: azure, region:
southafricanorth, tier: staging }
  azure-aks-prod:    { provider: azure-aks,      serviceConnection: ado-sc-azure-prod,   resource: aks-prod-01, cloud: azure, region:
southafricanorth, tier: prod }
  azure-aks-dr:      { provider: azure-aks,      serviceConnection: ado-sc-azure-dr,     resource: aks-dr-01,  cloud: azure, region:
southafricawest, tier: dr }
  azure-appsvc-staging:{ provider: azure-app-service, serviceConnection: ado-sc-azure-staging, resource: app-stg-01, cloud: azure, region:
southafricanorth, tier: staging }
  azure-appsvc-prod:  { provider: azure-app-service, serviceConnection: ado-sc-azure-prod,   resource: app-prod-01, cloud: azure, region:
southafricanorth, tier: prod }
  azure-appsvc-dr:    { provider: azure-app-service, serviceConnection: ado-sc-azure-dr,     resource: app-dr-01,  cloud: azure, region:
southafricawest, tier: dr }

  # — AWS (K8s & PaaS – Staging, Prod, DR) —
  aws-eks-staging:   { provider: aws-eks,        serviceConnection: ado-sc-aws-staging, resource: eks-stg-01, cloud: aws, region: eu-west-1,
tier: staging }
  aws-eks-prod:      { provider: aws-eks,        serviceConnection: ado-sc-aws-prod,    resource: eks-prod-01, cloud: aws, region: eu-west-1,
tier: prod }
  aws-eks-dr:        { provider: aws-eks,        serviceConnection: ado-sc-aws-dr,      resource: eks-dr-01,  cloud: aws, region: us-east-1,
tier: dr }
  aws-runner-staging: { provider: aws-app-runner, serviceConnection: ado-sc-aws-staging, resource: run-stg-01, cloud: aws, region: eu-west-1,
tier: staging }
  aws-runner-prod:    { provider: aws-app-runner, serviceConnection: ado-sc-aws-prod,    resource: run-prod-01, cloud: aws, region: eu-west-1,
tier: prod }
  aws-runner-dr:      { provider: aws-app-runner, serviceConnection: ado-sc-aws-dr,      resource: run-dr-01,  cloud: aws, region: us-east-1,
tier: dr }

  # — GCP (K8s & PaaS – Staging, Prod, DR) —
  gcp-gke-staging:   { provider: gcp-gke,        serviceConnection: ado-sc-gcp-staging, resource: gke-stg-01, cloud: gcp, region: europe-
west1, tier: staging }
  gcp-gke-prod:      { provider: gcp-gke,        serviceConnection: ado-sc-gcp-prod,    resource: gke-prod-01, cloud: gcp, region: europe-
west1, tier: prod }
  gcp-gke-dr:        { provider: gcp-gke,        serviceConnection: ado-sc-gcp-dr,      resource: gke-dr-01,  cloud: gcp, region: us-
central1, tier: dr }
  gcp-run-staging:   { provider: gcp-cloud-run,  serviceConnection: ado-sc-gcp-staging, resource: run-stg-01, cloud: gcp, region: europe-
west1, tier: staging }
  gcp-run-prod:      { provider: gcp-cloud-run,  serviceConnection: ado-sc-gcp-prod,    resource: run-prod-01, cloud: gcp, region: europe-
west1, tier: prod }
  gcp-run-dr:        { provider: gcp-cloud-run,  serviceConnection: ado-sc-gcp-dr,      resource: run-dr-01,  cloud: gcp, region: us-
```

```

centrall,      tier: dr }

# — DIGITALOCEAN (PaaS & K8s) —
do-app-staging: { provider: digitalocean-app, serviceConnection: ado-sc-do-staging, resource: do-app-stg, cloud: do, region: nycl,
tier: staging }
do-app-prod:    { provider: digitalocean-app, serviceConnection: ado-sc-do-prod,   resource: do-app-prod, cloud: do, region: nycl,
tier: prod }
do-doks-dr:     { provider: digitalocean-doks, serviceConnection: ado-sc-do-dr,    resource: do-doks-dr,  cloud: do, region: sfo2,
tier: dr }

```

Adding a new target = 3 things (not just a line):

1. Infrastructure in Target Cloud → provisioned via Terraform (providers/compute/<target>/)
2. ADO service connection → created once in Azure DevOps
3. Registry entry (PR) → appended to targets.yml, validated by targets-ci.yml

Guardrails that keep it safe as it grows:

targets-ci.yml (on PR to targets.yml) ✓ schema valid / unique names / valid tier/region ✓ service connection exists (ADO REST) ✓ target resource exists in Cloud (az/aws/gcloud/doctl) ✓ no orphaned catalog references on removal	
catalog-ci.yml cross-check ✓ every app's deploymentTarget: value exists in targets.yml ✓ every app's secretsProvider: value exists in secrets.yml	
Platform-only RBAC ✓ only platform team merges target-registry	

What About Cost? (Multi-cloud billing handled via dynamic cost tags) | Cloud | Cost Tagging Mechanism | |
 —|—| | Azure | Resource tags on all Azure resources | | AWS | AWS Cost Allocation Tags | | GCP | GCP Labels +
 Billing export to BigQuery | The platform-policy.yml automatically applies the right tag schema per cloud,
 and the **Cloud Engineering Dashboard** aggregates costs across all clouds by department/app.

Target Type decision (\$20 cost & complexity): | Isolation / Workload need | Use | |—|—| | Microservices /
 complex networking | **Kubernetes (AKS/EKS/GKE/DOKS)** | | Standard Web Apps / APIs (No K8s overhead) |
PaaS (App Service, App Runner, Cloud Run, DO App) | | Mobile backend / Auth / Realtime DB | **BaaS**
(Firebase/Supabase/Appwrite) | | Event-driven / lightweight tasks | **Serverless (Lambda, Cloud**
Functions, Azure Functions) | | Static sites / CMS / simple backups | **VMs**
(DigitalOcean/Hostinger/Kamatera) |

DIAGRAM 8 — Concurrency (The Two Scenarios)

Scenario 1: 10 PRs on the SAME repo (ds-internal/canteen)

PR #1 ... PR #10 → 10 INDEPENDENT validation runs (parallel, limited by agents)

EACH run (triggered on feature branch or develop branch):

- repo-contract-check (reads repo – no contention)
- reads the SAME catalog entry (read-only → no contention)
- builds/tests its own feature branch
- publishes results tagged by PR/build id (no collision)
- builds artifact with UNIQUE tag (no registry overwrite)
- deploys to ISOLATED ephemeral preview environment (validate mode)
- none touch shared staging/prod • DR never triggers (prod-only)

↓

MERGE into develop branch is SEQUENTIAL (git advances one merge at a time)

- PRs merge one at a time
- if PR #3 & #7 touch same file → later one conflicts → rebase
- branch policy on develop branch: build validation + reviewers before merge
- concurrency: cancel-in-progress-runs=true
 - only newest develop branch build proceeds (avoids deploy storm)

↓

PRs from develop branch to main branch:

- one PR at a time per branch policy
- each has 24-hour approval window
- approval is manual (never auto-approved)

Safe because: validate mode + read-only catalog + unique tags + concurrency controls.

Scenario 2: 10 PRs across DIFFERENT repos/projects

```

ds-internal/canteen → run 1 → own catalog entry
ds-internal/asset-register → run 2 → own catalog entry
ds-external/hack-2026-team-1 → run 3 → own catalog entry

```

```

... (10 repos)
↓
10 FULLY INDEPENDENT, PARALLEL runs
• each reads its OWN catalog entry (no shared file)
• each routes to its OWN goldenPath + template library
• each targets its OWN deployment target (AKS, App Runner, Firebase, DO, etc.)
• each publishes its OWN results to Grafana
• each uses its OWN secrets provider namespace
• no shared branch, environment, or catalog
↓
ONLY LIMITER: AGENT POOL CAPACITY + TARGET API RATE LIMITS
(fewer agents than runs → extras queue, then run)

```

Safe because: no coupling at all — just agent capacity and API limits.

DIAGRAM 9 — Template Selection + Wrong-Template Handling

How the developer chooses (no detection → guidance must be excellent)

Onboarding presents GOLDEN PATH CATALOG (_catalog.json)
Each card shows: What / Use when / Don't use when / Examples / You get
↓
DECISION TREE (fast path)
What are you building?
— User-facing web app or API? → web-api (AKS / EKS / GKE / App Runner / Cloud Run / App Service)
— Mobile app backend (Auth/DB)? → mobile-baas (Firebase / Supabase)
— Serve an ML model as an API? → ai-ml (Tier 1)
— TRAIN an ML model (GPUs)? → ai-ml-training (Tier 2)
— Report / dashboard / data pipeline? → data-bi
— Business Central extension / AL? → business-central
— Multi-cloud DR backup? → backup-system (DigitalOcean / AWS)
— Static site / CMS / WordPress? → static-site (Hostinger / SiteGround)
— Azure infra / Terraform? → infrastructure
— App around Business Central? → peripheral-software
— Streaming / edge (dedicated infra)? → streaming / edge (Tier 2)
— Doesn't fit any? → Tier 3 escape hatch

Standard vs. Specialized — how you tell

```

meta.yaml per template:
web-api:      { tier: 1, type: standard,   requiresDedicatedInfra: false }
mobile-baas:  { tier: 1, type: standard,   requiresDedicatedInfra: false, usesK8s: false }
ai-ml-training: { tier: 2, type: specialized, requiresDedicatedInfra: true,
               provisionOnDemand: true, teamSuppliesConfig: [gpuType,...] }

requiresDedicatedInfra: true = specialized (Tier 2)
usesK8s: false              = BaaS / PaaS / Serverless target

```

Wrong template / needs improvement

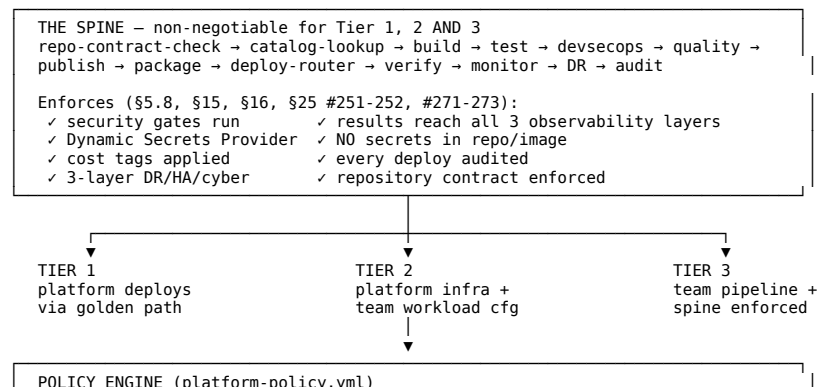
WRONG, realised EARLY (before 1st deploy)
→ re-run scaffold, pick correct path, old entry replaced (PR)

WRONG, realised LATE (build fails / runtime off)
→ build error names the mismatch (\$258)
→ "Switch Golden Path" action updates catalog goldenPath via PR

RIGHT, but THIS APP needs more
→ add app-level override in the catalog entry's config:

RIGHT, but the TEMPLATE itself needs improvement
→ PR to golden-path-templates → platform reviews
→ template VERSION bumps (semver) → apps OPT IN

DIAGRAM 10 — The Spine + Policy Engine + Separated Observability (All 7 Disciplines)



catalog (6 fields)	+	policy rules	→	full manifest
department		departments:		owner
criticality	→	peripheral:		target{staging,prod,dr}
goldenPath		owner:		ICT-...
deploymentTarget		projects:		observabilityProvider
secretsProvider		ds-internal:		replicas/resources
observabilityProvider		tier:		prod failover
		clusters:...		templateLibrary
		criticality:		cost tags (Azure tags / AWS tags / GCP
		high: {replicas:3,...}		labels applied dynamically)
		goldenPaths:		workloadIdentity (if K8s: Entra/IRSA/GKE)
		web-api: {template: api}		
		mobile-baas: {template: firebase-app}		

Change a rule once → every app inherits it on next run

OBSERVABILITY – THREE SEPARATED CONCERNS (§9 – NOT Grafana alone)

1. OPERATIONAL OBSERVABILITY (Grafana + Prometheus + Cloud Monitors)
publish-results.yml exports engineering metrics
Tagged by: app, department, goldenPath, tier, target, run-id



PLATFORM ENGINEERING DASHBOARD

- Golden path usage, scaffold success rate
- Time from scaffold to first deploy
- Self-service adoption metrics



DEVOPS DASHBOARD

- Build success rate, deploy frequency, lead time
- Pipeline duration, 24h approval timeout rate



DEVSECOPS DASHBOARD

- Vulnerabilities by severity, gate pass/fail per app
- Secret leak attempts, container vulns, SBOM compliance
- Time to remediation



INFRASTRUCTURE ENGINEERING DASHBOARD

- Terraform success rate, drift detection
- Target utilization (CPU/memory per AKS, EKS, GKE, DO)



CLOUD ENGINEERING DASHBOARD

- Multi-cloud resource consumption by department
- Cost attribution (aggregates Azure, AWS, GCP billing)
- Unused resources, oversized workloads, budget alerts



SRE DASHBOARD

- SLI/SLO compliance, error budget burn, uptime
- MTTR, incident frequency
- DR failover test results, RTO/RPO compliance
- Cyber-resilience status (last backup, last recovery test)

2. DEVELOPER FEEDBACK (Azure DevOps native – not Grafana)

- PR comments with gate results + actionable remediation steps
- Pipeline summary page (build/test/security/quality status)
- ADO Work Items auto-created for failed gates (assign to owner)
- Teams/email notifications (per-app, routed to owner group)
- Wiki links embedded in failure messages

This is what developers see day-to-day (NOT Grafana).

3. SECURITY GOVERNANCE (Microsoft security stack – not Grafana)

- Microsoft Defender for DevOps – pipeline security posture
- Microsoft Defender for Cloud – Azure resource security
- Microsoft Defender for Containers – runtime container threat
- Microsoft Sentinel – SIEM (correlate incidents)
- Azure Policy – continuous compliance enforcement
- Entra ID audit logs – identity + access events
- Secret Provider audit logs (KV/Vault/AWS SM access events)

This is what ICT-Governance / Security team sees.



ALERTING RULES (all three layers)

- Security: critical vuln → ICT-Governance via Defender
- Reliability: SLO breach → app owner via Teams (operational)
- Cost: budget exceeded → department lead (operational)
- DR: replica unhealthy → Platform team (operational)
- Approval: 24h timeout warning → app owner at 20h mark (developer feedback)

DIAGRAM 11 — Build Phasing (Tier 1 → Tier 2 → Tier 3 + Portal Evolution + Multi-Cloud)

PHASE 0 – Landing zone

Terraform: subscription, VNet, ACR, Key Vault, Entra groups

AKS clusters: aks-test-01, aks-prod-01, aks-dr-01, aks-sandbox-01

Service connections: ado-sc-repo-creator, ado-sc-github-sync, gh-sc-hackathon-setup

Workload Identity infrastructure: OIDC issuer enabled on all AKS clusters

PHASE 1 – Tier 1 core + External flow ← build first (PowerShell era)
Internal onboarding: onboard-app.ps1 + Terraform (ado-repo module)
External onboarding: onboard-hackathon.ps1 + Terraform (github-repo + ado-repo modules)
golden-path-templates (web-api first) + _catalog.json + decision tree
platform-policy.yml + catalog-schema.json + catalog-ci.yml
app-dispatcher.yml + spine (repo-contract-check → ci-base → gates → package → deploy-router)
Workload Identity + Key Vault integration (default for K8s apps)
target-registry + targets-ci.yml (with staging/prod/dr environments mapped)
24-hour promotion window (PR develop branch → main branch → prod → auto-cancel on timeout)
One-way sync service (GitHub main branch → ADO develop branch)
Repository contract: required/allowed/forbidden enforced
PILOT: scaffold internal app (canteen) + onboard hackathon team (hack-2026-team-1)

PHASE 2 – DevSecOps + observability hardening
full gate suite · publish-results → 3 observability layers · alerting
All 7 discipline dashboards in Grafana (operational)
ADO PR comments + Work Items (developer feedback)
Microsoft Defender for DevOps/Cloud/Containers + Sentinel (governance)

PHASE 3 – SRE + DR + cyber-resilience
monitor-register · SLI/SLO · dr-provision (HA + DR + cyber-resilience) · dr-teardown
Immutable backups · quarterly recovery tests · Global routing failover

PHASE 4 – Expand Providers (Multi-Cloud, BaaS & PaaS) ← DYNAMIC EXPANSION
PHASE 4a: AWS EKS + App Runner + AWS Secrets Manager (Add when you have AWS workloads)
PHASE 4b: GCP GKE + Cloud Run + GCP Secret Manager (Add for GCP-specific services)
PHASE 4c: Multi-cloud DR (Azure ↔ AWS ↔ GCP failover routing)
Add providers/compute/digitalocean-app (for BC backup DR scenario)
Add providers/compute/firebase & supabase (for mobile-baas Golden Path)
Add providers/secrets/hashicorp-vault (enterprise secrets)
Add providers/compute/kamatera-vm & hostinger (for legacy/static sites)
Remaining Tier 1 paths + Tier 2 framework
specialized/ templates + on-demand infra (aks-gpu-01 etc.)

PHASE 5 – Tier 3 escape hatch ← code later
register-custom-app.ps1 + escape-hatch.yml + contract-verify.yml
approval workflow · dedicated/isolated cluster · re-review
(built only if Tier 3 demand materializes)

PHASE 6 – Platform API + Developer Portal ← end-state (post-Phase 3)
Expose scaffold/catalog/target-registry as REST API
Build Developer Portal (React/Blazor):

Create Application

Department: [AI Sizing ▼]

Application type:[mobile-baas▼]

Deploy Target: [Firebase ▼]

Secrets Provider:[GCP SM ▼]

Observability: [Datadog ▼]

Criticality: [High ▼]

App name: [_____]

[Create]

→ Platform provisions everything

→ Repo created

→ Pipeline configured

→ Dashboard created

→ App URL generated

PowerShell scripts kept as fallback / CI automation
This is when it becomes a true INTERNAL DEVELOPER PLATFORM

Summary — Everything Captured (Ultimate God Mode + Dynamic Providers + Multi-Cloud)

Topic	Where it lives
Very beginning = onboarding PowerShell scripts (internal + external)	Diagrams 3, 4
Service connections create repos (Basic license users can't)	Diagrams 2, 3, 4
Internal scaffold (no detection)	Diagram 3
External hackathon onboarding (GitHub + ADO, one-way sync)	Diagram 4
Catalog generated by scaffold (T1/T2) / registration (T3)	Diagrams 3, 4, 6
Catalog lives in app-catalog, not app repo	Diagrams 2, 5
Repository contract (required/allowed/forbidden)	Diagrams 2, 5
git push → production flow (internal + external)	Diagram 5

Topic	Where it lives
24-hour approval window (auto-CANCEL if not approved — never auto-approve)	Diagram 5
Main branch is source of truth (PR from develop branch → main branch → prod)	Diagrams 2, 3, 4, 5
Branch model: only develop branch + main branch exist	Diagrams 2, 3, 4
One-way sync service (GitHub → ADO, never back)	Diagram 4
Dynamic Deployment Target Registry + targets-ci + guardrails	Diagram 7
Target vs Namespace vs BaaS decision (cost \$20)	Diagram 7
Multi-Cloud Cost Tagging (Azure tags, AWS tags, GCP labels)	Diagram 7
Concurrency: same repo / different repos	Diagram 8
3 mature tiers (T1 & T2 now, T3 code later)	Diagrams 2, 6, 11
Policy engine + universal spine	Diagram 10
Observability in 3 separated layers (operational/developer/governance)	Diagram 10
Dynamic Secrets Provider Abstraction (Azure KV, HashiCorp, AWS SM, etc.)	Diagrams 5, 6, 10
Dynamic Observability Provider (Azure Monitor, CloudWatch, Cloud Logging)	Diagrams 1, 5, 6, 10
Multi-Cloud Routing Mechanics (Same Spine, Provider Plugins)	Diagram 5
Concrete Catalog Examples (Same App across Azure, AWS, GCP)	Diagram 6
Provider Plugin Architecture (Folder structure for EKS/GKE/PaaS)	Diagram 2
Security Model per Cloud (Workload Identity / IRSA / GKE WI / Managed ID)	Diagram 5
Multi-Cloud DR Scenario (Azure Primary → AWS DR)	Diagram 6
Non-K8s workloads supported (Firebase, Supabase, Amplify, PaaS, VMs)	Diagrams 1, 5, 6, 9
Modular Provider Plugin Architecture (Extensibility)	Diagram 2, 11
DR expanded: HA + DR + cyber-resilience (immutable backups)	Diagrams 5, 11
Template selection + wrong-template handling	Diagram 9
DR / failover / teardown	Diagram 5
Developer Portal as end-state (Phase 6)	Diagram 11
All 7 engineering disciplines publish results	Diagram 10
Staging, Prod, DR mapped for ALL targets in Registry	Diagram 7
Kubernetes is strictly optional per cloud (PaaS/Serverless supported)	Diagrams 1, 2, 5, 6, 7

Final Corrections Checklist (All 14 Expert Review Points Addressed)

1. ☒ **Branch model is standard:** develop branch → PR → main branch → production (NOT “main created after production”)
 2. ☒ **24-hour window, NOT auto-approval:** explicitly called “24-hour approval window with automatic cancellation on timeout” — timeout NEVER means approval
 3. ☒ **Repository contract** instead of strict “src/+tests/ only”: required / allowed / forbidden lists, allows docs/migrations/language manifests, forbids pipeline files/secrets/.env/k8s manifests/Helm
 4. ☒ **Dynamic Secrets Provider Abstraction:** catalog declares secretsProvider (Azure KV, HashiCorp, AWS SM, CyberArk, Akeyless). Platform routes to correct provider module
 5. ☒ **DR = 3 layers:** High Availability (replicas/zones/HPA) + Disaster Recovery (secondary region/geo-replication/Global routing) + Cyber-resilience (immutable backups/WORM/isolated access/quarterly recovery tests)
 6. ☒ **Developer portal evolution path:** PowerShell (Phase 1) → Platform API (Phase 2) → Developer Portal UI (Phase 6)
 7. ☒ **Observability NOT Grafana alone:** explicitly separated into Operational (Grafana+Prometheus+Cloud Monitors), Developer feedback (ADO PR/Work Items/Teams), Security governance (Defender+Sentinel+Policy+Entra+KV audit logs)
 8. ☒ **Dynamic Deployment Target Abstraction:** catalog declares deploymentTarget (AKS, EKS, GKE, DigitalOcean, Firebase, Supabase, AWS Amplify, Kamatera, Hostinger, etc.). Supports K8s and non-K8s
 9. ☒ **Provider-agnostic deploy step:** Pipeline routes to native CLIs/APIs (firebase deploy, supabase db push, doctl) based on target. Mobile apps and static sites fully supported
 10. ☒ **Modular Provider Plugin Architecture:** New tech stack = add folder to providers/compute/ or providers/secrets/. Core Spine never changes. Freedom of choice for any tech stack
 11. ☒ **Branch naming explicit:** “develop branch” and “main branch” used everywhere, not just “develop” and “main”. Only 2 branches exist
 12. ☒ **All 11 diagrams + summary section included:** nothing omitted
 13. ☒ **Multi-Cloud explicitly detailed & fused:** AWS EKS, GCP GKE, IRSA, Workload Identity, Cross-Cloud DR, Dynamic Observability, and Multi-Cloud Cost Tagging fully integrated into the core diagrams and examples without breaking the architecture flow.
 14. ☒ **Environment Parity & K8s Optional:** Staging, Prod, and DR are explicitly mapped for ALL targets. Kubernetes is strictly optional — AWS App Runner, GCP Cloud Run, and Azure App Service are fully supported alongside their K8s counterparts via identical pipeline spines.
-

Build Order (Your Phase 1 First Deliverables)

The first concrete deliverables that prove the platform works end-to-end:

1. onboard-app.ps1 (internal scaffold – PowerShell + Terraform)
2. onboard-hackathon.ps1 (external scaffold – GitHub + ADO + sync)
3. Terraform: ado-repo + github-repo modules (service-connection-backed)
4. web-api/template.json (first Golden Path)
5. catalog.json (front door decision tree)
6. platform-policy.yml (derivation brain)
7. catalog-schema.json + catalog-ci.yml (validation)
8. app-dispatcher.yml + _shared/*.yml (the spine + deploy-router)
9. target-registry/targets.yml + targets-ci.yml (staging/prod/dr mapping)
10. providers/compute/azure-aks/ + providers/secrets/azure-key-vault/ (first plugins)
11. 24-hour promotion gate + one-way sync service
12. Grafana operational dashboards (Phase 2)
13. Microsoft Defender + Sentinel integration (Phase 2)
14. providers/compute/aws-eks/ + providers/compute/aws-app-runner/ + providers/secrets/aws-secrets-manager/ (Phase 4 plugins)
15. providers/compute/gcp-gke/ + providers/compute/gcp-cloud-run/ + providers/secrets/gcp-secret-manager/ (Phase 4 plugins)

Pilot: scaffold canteen (internal) + hack-2026-team-1 (external) end-to-end through this platform.

This is the **Ultimate God Mode (Dynamic, Modular & Multi-Cloud)** version.